

Kinematics of Differential Drive Robots and Odometry

Supervisor:

Dr. Khaled Youssef

Prepared by:

Mustafa Ahmed Abdullah

Marwan Tamer Fathy

College of Navigation Science and Space Technology (NNST)

1 Introduction

The fundamental principle of a differential drive robot is that the difference between the RPM of the right and left wheels causes the robot to move forward, backward, or steer.

The state of the robot is defined by a pose vector representing its position and orientation in the 2D plane:

$$X_{Robot} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (1)$$

The kinematic state equations relating the linear velocity (v) and angular velocity (ω) to the global pose are defined as:

$$\dot{x} = v \cos(\theta) \quad (2)$$

$$\dot{y} = v \sin(\theta) \quad (3)$$

$$\dot{\theta} = \omega \quad (4)$$

This can be written in matrix form as:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} \quad (5)$$

2 Rigid Body Kinematics (Transport Theorem)

To find the velocity of the center of the robot (C), we apply the Transport Theorem for rigid body kinematics from both the right wheel (Point A) and the left wheel (Point B).

Let $2b$ be the distance between the two wheels.

2.1 Linear Velocity Derivation

The velocity of the center C can be expressed from A and B as:

$$\vec{V}_c = \vec{V}_A + \vec{\omega} \times \vec{r}_{C/A} \quad (6)$$

$$\vec{V}_c = \vec{V}_B + \vec{\omega} \times \vec{r}_{C/B} \quad (7)$$

Adding the two equations yields:

$$2\vec{V}_c = \vec{V}_A + \vec{V}_B + \vec{\omega} \times (\vec{r}_{C/A} + \vec{r}_{C/B}) \quad (8)$$

Since the center C is exactly midway between A and B , the vectors are equal and opposite ($\vec{r}_{C/A} = -\vec{r}_{C/B}$). Therefore, the cross product term cancels out:

$$2\vec{V}_c = \vec{V}_A + \vec{V}_B \quad (9)$$

$$\vec{V}_c = \frac{\vec{V}_A + \vec{V}_B}{2} \quad (10)$$

Assuming rolling without slipping, the velocity of each wheel is related to its angular velocity and radius by $v = \dot{\phi}r$.

Substituting this in:

$$V_c = \frac{r_R\dot{\phi}_R + r_L\dot{\phi}_L}{2} \quad (11)$$

2.2 Angular Velocity Derivation

Instead of adding the equations, we subtract them to solve for the angular velocity ω :

$$0 = \vec{V}_A - \vec{V}_B + \vec{\omega} \times (\vec{r}_{C/A} - \vec{r}_{C/B}) \quad (12)$$

Notice that $(\vec{r}_{C/A} - \vec{r}_{C/B}) = \vec{r}_{B/A}$ which is the vector pointing from wheel A to wheel B with a magnitude of $2b$:

$$\vec{V}_A - \vec{V}_B = -\vec{\omega} \times \vec{r}_{B/A} \quad (13)$$

Using a correct right-handed system (where ω is out of the plane and the distance vector is in the plane), their cross product simplifies to the multiplication of their magnitudes:

$$v_R - v_L = 2b \cdot \omega \quad (14)$$

$$r_R\dot{\phi}_R - r_L\dot{\phi}_L = 2b \cdot \omega \quad (15)$$

Solving for ω :

$$\omega = \frac{r_R\dot{\phi}_R - r_L\dot{\phi}_L}{2b} \quad (16)$$

3 Differential Forward Kinematics

From the derived linear and angular velocities, we can go directly from joint velocities (wheel angular velocities) to workspace velocities (global pose derivatives). This is represented by the Jacobian Matrix mapping the inputs to the state derivatives:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r_R \cos(\theta)}{2} & \frac{r_L \cos(\theta)}{2} \\ \frac{r_R \sin(\theta)}{2} & \frac{r_L \sin(\theta)}{2} \\ \frac{r_R}{2b} & -\frac{r_L}{2b} \end{bmatrix} \begin{bmatrix} \dot{\phi}_R \\ \dot{\phi}_L \end{bmatrix} \quad (17)$$

3.1 Kinematic States and Observations

Based on the derived kinematics, the following operational states are observed:

1. **Forward Pure Translation:** If $\dot{\phi}_R = \dot{\phi}_L = \dot{\phi} > 0$
 $V_c = V_A = V_B = \dot{\phi}r$
2. **Backward Pure Translation:** If $\dot{\phi}_R = \dot{\phi}_L = -\dot{\phi} < 0$
 $V_c = V_A = V_B = -\dot{\phi}r$
3. **Forward and Left Turn:** If $\dot{\phi}_R > \dot{\phi}_L > 0$
4. **Forward and Right Turn:** If $0 < \dot{\phi}_R < \dot{\phi}_L$
5. **Pure Rotation (Spin in place):** Assuming equal wheel radii ($r_R = r_L$), this occurs when $V_A = -V_B$ (or $\dot{\phi}_R = -\dot{\phi}_L$).

4 MATLAB Simulation Code

Below is the exact implementation of the kinematics integration and visualization to simulate the differential drive robot.

```
clear; clc; close all;
% =====
% 1. ROBOT CAR AND TRACK PARAMETERS
% =====
rR = 0.0325;      % Radius of Right wheel (meters)
rL = 0.0325;      % Radius of Left wheel (meters)
b = 0.1;         % Half-width of the robot (meters)
v = 0.3;         % Target forward velocity (m/s)

% Track dimensions based on the provided image
L_side = 1.0;     % Length of vertical sides (100cm)
L_bottom = 1.0;   % Length of bottom side (100cm)
R_turn = 0.5;     % Radius of the top semi-circle (50cm)

% Starting point (Middle of the bottom line)
start_x = 0.5;    % x-coordinate (exactly in the middle of the 1m line)
start_y = 0.0;    % y-coordinate
start_theta = 0;  % Initial heading (0 rad = facing right)

% Spin velocity for sharp 90-degree corners (rad/s)
omega_spin = pi/4;

% =====
% 2. TIMING AND COMMAND SIGNALS
% =====
% The track consists of 7 segments for one complete lap:
t_seg = zeros(1, 7);
t_seg(1) = (1.0 - start_x) / v;      % 1. Straight right to bottom-right corner
t_seg(2) = (pi/2) / omega_spin;      % 2. Point turn 90 deg CCW
t_seg(3) = L_side / v;               % 3. Straight up the right side
t_seg(4) = (pi * R_turn) / v;        % 4. Top semi-circle U-turn
t_seg(5) = L_side / v;               % 5. Straight down the left side
t_seg(6) = (pi/2) / omega_spin;      % 6. Point turn 90 deg CCW
t_seg(7) = start_x / v;              % 7. Straight right back to the start mark

t_end = cumsum(t_seg);               % End times for each segment
t_lap = t_end(end);                  % Total time for one complete lap

dt = 0.003;
t = 0:dt:(2 * t_lap);               % Run for exactly 2 full laps

% Pre-allocate wheel velocities
phidotR = zeros(1, length(t));
phidotL = zeros(1, length(t));

% Fill the arrays with commands based on the track shape
for i = 1:length(t)
    time_in_lap = mod(t(i), t_lap);

    if time_in_lap <= t_end(1)
        % Segment 1: Straight right
        phidotR(i) = v / rR;
        phidotL(i) = v / rL;
    elseif time_in_lap <= t_end(2)
        % Segment 2: Spin 90 deg CCW in place
        phidotR(i) = omega_spin * b / rR;
        phidotL(i) = -omega_spin * b / rL;
    elseif time_in_lap <= t_end(3)
        % Segment 3: Straight up
        phidotR(i) = v / rR;
        phidotL(i) = v / rL;
    elseif time_in_lap <= t_end(4)
        % Segment 4: U-turn (not explicitly shown in the code block, but implied by the track shape)
        % This segment is covered by the semi-circle U-turn in the timing array.
    end
end
```

```

        % Segment 4: Top Semi-circle
        omega_curve = v / R_turn;
        phidotR(i) = (v + omega_curve * b) / rR;
        phidotL(i) = (v - omega_curve * b) / rL;
    elseif time_in_lap <= t_end(5)
        % Segment 5: Straight down
        phidotR(i) = v / rR;
        phidotL(i) = v / rL;
    elseif time_in_lap <= t_end(6)
        % Segment 6: Spin 90 deg CCW in place
        phidotR(i) = omega_spin * b / rR;
        phidotL(i) = -omega_spin * b / rL;
    else
        % Segment 7: Straight right to complete lap
        phidotR(i) = v / rR;
        phidotL(i) = v / rL;
    end
end
end

% =====
% 3. KINEMATICS INTEGRATION
% =====
x = zeros(1, length(t));
y = zeros(1, length(t));
theta = zeros(1, length(t));

% Apply the starting positions
x(1) = start_x;
y(1) = start_y;
theta(1) = start_theta;

for ii = 2:length(t)
    J = [rR*cos(theta(ii-1))/2, rL*cos(theta(ii-1))/2; ...
        rR*sin(theta(ii-1))/2, rL*sin(theta(ii-1))/2; ...
        0.5*rR/b, -0.5*rL/b];
    deltapose = dt * J * [phidotR(ii-1); phidotL(ii-1)];
    x(ii) = x(ii-1) + deltapose(1);
    y(ii) = y(ii-1) + deltapose(2);
    theta(ii) = theta(ii-1) + deltapose(3);
end

% =====
% 4. VISUALIZATION & ANIMATION
% =====
figure('Name', 'Window Shaped Track Animation', 'Position', [100, 100, 600, 700]);
hold on; axis equal; grid on;
title('Differential Drive Robot on Window Track');
xlabel('X (meters)'); ylabel('Y (meters)');
xlim([-0.4, 1.4]); ylim([-0.4, 1.8]);

% Draw the new track
plot([0, 1], [0, 0], 'k--', 'LineWidth', 2); % Bottom straight
plot([1, 1], [0, 1], 'k--', 'LineWidth', 2); % Right straight
plot([0, 0], [1, 0], 'k--', 'LineWidth', 2); % Left straight
ang = linspace(0, pi, 50);
plot(0.5 + 0.5*cos(ang), 1 + 0.5*sin(ang), 'k--', 'LineWidth', 2); % Top semi-circle

% Draw the start mark from the middle of the bottom line
plot([start_x start_x], [-0.05 0.05], 'r-', 'LineWidth', 2);
text(start_x, -0.1, 'Start (Middle)', 'Color', 'r', 'HorizontalAlignment', 'center');

% Define robot shape
chassis_x = [-0.1, 0.1, 0.1, -0.1];
chassis_y = [-0.1, -0.1, 0.1, 0.1];
wheel_x = [-0.08, 0.08, 0.08, -0.08];
wheelR_y = [-0.17, -0.17, -0.11, -0.11];
wheelL_y = [0.11, 0.11, 0.17, 0.17];
h_chassis = patch('XData', [], 'YData', [], 'FaceColor', [0.7 0.7 0.7], 'EdgeColor', 'k');

```

```

h_wheelR = patch('XData', [], 'YData', [], 'FaceColor', 'k');
h_wheelL = patch('XData', [], 'YData', [], 'FaceColor', 'k');
h_nose   = plot(0, 0, 'r-', 'LineWidth', 3);

% Animation loop
for ii = 1:10:length(t)
    th = theta(ii);
    R_mat = [cos(th), -sin(th); sin(th), cos(th)];

    rot_chassis = R_mat * [chassis_x; chassis_y];
    rot_wheelR = R_mat * [wheel_x; wheelR_y];
    rot_wheelL = R_mat * [wheel_x; wheelL_y];

    set(h_chassis, 'XData', rot_chassis(1,:) + x(ii), 'YData', rot_chassis(2,:) + y(ii));
    set(h_wheelR, 'XData', rot_wheelR(1,:) + x(ii), 'YData', rot_wheelR(2,:) + y(ii));
    set(h_wheelL, 'XData', rot_wheelL(1,:) + x(ii), 'YData', rot_wheelL(2,:) + y(ii));
    set(h_nose, 'XData', [x(ii), x(ii) + 0.15*cos(th)], 'YData', [y(ii), y(ii) + 0.15*sin(th)]);

    drawnow;
end
fprintf('Finished simulating exactly %d points.\n', length(t));

```